

Security Finding Report

FalconStrix – Insufficient Session Expiration on Logout

Report ID	FALSTRX-2026-008
Affected Component	Session management (Flask signed-cookie sessions); /logout route
Vulnerability Class	Insufficient Session Expiration – CWE-613
Severity	High — CVSS 3.1: 8.1 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N)
Discovered By	Manual black-box testing (Burp Suite), cookie replay after logout
Environment	Isolated local lab – Windows 11 target, Kali Linux (VMware) attacker
Assessment Date	28–29 July 2026
Status	Fixed & Verified

1. Summary

FalconStrix's "Sign Out" action cleared the session cookie client-side but did not invalidate the underlying session on the server. A session cookie captured before logout continued to authenticate successfully — including performing real, state-changing actions and downloading reports — indefinitely after the legitimate user had logged out, until the cookie's natural 12-hour expiry.

2. Root Cause

FalconStrix uses Flask's default client-side signed-cookie sessions: all session data (user ID, username, role) is encoded and cryptographically signed directly inside the cookie, with no corresponding server-side record of which sessions are currently valid. The /logout route only cleared the session on the client (`session.clear()`) and issued a new, empty cookie in the response — it had no mechanism to mark a previously issued cookie as revoked, since no server-side state existed to revoke in the first place.

3. Steps to Reproduce

- Log into FalconStrix normally and capture the session cookie value from any successful authenticated request in Burp.
- Click "Sign Out" in the browser as a normal user would.
- In Burp Repeater, replay a request using the old, pre-logout cookie value against protected routes.

Evidence – viewing the dashboard after logout:

```
GET /dashboard HTTP/1.1
Cookie: session=<cookie captured before logout>
```

--- Response ---

```
HTTP/1.1 200 OK
... full dashboard HTML returned ...
```

Evidence – performing a real action (creating an alert) after logout:

```
POST /api/alerts/manual HTTP/1.1
Cookie: session=<same pre-logout cookie>
{"severity":"HIGH","message":"Manual operator alert from zaid at 5:18:25 AM"}

--- Response ---
HTTP/1.1 200 OK
{"alert_id":313, "ok":true, ...}
```

Evidence – downloading a PDF report after logout:

```
GET /api/reports/resolved-cases.pdf HTTP/1.1
Cookie: session=<same pre-logout cookie>

--- Response ---
HTTP/1.1 200 OK
Content-Disposition: attachment; filename=FalconStrix_Resolved_Cases.pdf
%PDF-1.4 ... (real, valid PDF content)
```

All three requests succeeded using a cookie captured before the logout action, confirming the session remained fully authenticated — for both reading data and performing real, state-changing actions — after the user had logged out.

4. Impact

- Any party who obtains a copy of a user's session cookie (via a shared/compromised machine, network capture, browser history/cache, or a separate vulnerability such as XSS) retains full account access indefinitely after the legitimate user believes they have logged out.
- The impact is not limited to passive viewing: the captured session could create alerts, generate reports, and perform other state-changing actions as the logged-out user.
- Users have no reliable way to end their own session early (e.g. on a shared or public computer) — clicking “Sign Out” provides a false sense of security.

5. Severity Justification

Factor	Rating	Reasoning
Attack Vector	Network	Exploitable by anyone possessing a copy of the session cookie.
Attack Complexity	Low	Simply replaying a previously captured cookie value.
Privileges Required	None (beyond possessing the cookie)	No additional credentials needed, only the captured cookie.
User Interaction	None	No action needed from the victim after the cookie is obtained.
Confidentiality Impact	High	Full account data readable indefinitely post-logout.
Integrity Impact	High	Real state-changing actions (alert creation) succeed post-logout.

Estimated CVSS 3.1 vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N → Base Score **8.1 (High)**.

6. Remediation

A server-side session version counter was introduced, giving the application a real mechanism to revoke previously issued cookies without requiring a full server-side session store.

6.1 Database: per-user session version column

```
ALTER TABLE Users ADD COLUMN session_version INT NOT NULL DEFAULT 1;
```

6.2 Include the version in the authenticated user record (auth_service.py)

```
row = fetch_query(
    """SELECT user_id, username, role, password_hash, session_version
    FROM Users WHERE username = %s""",
    (username.strip(),),
    fetchall=False,
)
...
if check_password_hash(row['password_hash'], password):
    return {
        'user_id': row['user_id'],
        'username': row['username'],
        'role': row.get('role') or 'user',
        'session_version': row.get('session_version', 1),
    }
```

6.3 Stamp the version into the session at login (app.py)

```
session.permanent = True
session['user_id'] = user['user_id']
session['username'] = user['username']
session['role'] = user.get('role', 'user')
session['session_version'] = user.get('session_version', 1)
```

6.4 Validate the version on every request (app.py, before_request)

```
if not session.get('user_id'):
    return redirect(url_for('login', next=request.path))

current_version_row = fetch_query(
    "SELECT session_version FROM Users WHERE user_id = %s",
    (session.get('user_id'),),
    fetchall=False,
)
current_version = current_version_row.get('session_version') if current_version_row else None
if current_version is None or current_version != session.get('session_version'):
    session.clear()
    return redirect(url_for('login', next=request.path))
```

6.5 Bump the version on logout (app.py)

```
if uid:
    execute_query(
        "UPDATE Users SET session_version = session_version + 1 WHERE user_id = %s",
        (uid,),
```

```
)
session.clear()
return redirect(url_for('login'))
```

In plain terms: every issued session cookie carries a version number. Every request compares that number against the current value stored for the user in the database. Logout increments that stored value by one, which instantly invalidates every previously issued cookie for that user — regardless of how many copies exist or where they are held — on their very next use.

7. Verification (Post-Fix Re-Testing)

The original reproduction steps were repeated unmodified: capture a session cookie, log out, then replay the old cookie against /dashboard.

Test Condition	Before Fix	After Fix
Old cookie replayed against GET /dashboard	200 OK — full dashboard returned	Redirected to /login (session version mismatch)
Old cookie replayed against POST /api/alerts	200 OK — real alert created	Rejected — session invalidated
Old cookie replayed against PDF report endpoint	200 OK — valid PDF returned	Rejected — session invalidated
Legitimate login and normal use	Working	Still working — new session version issued correctly at login

8. Testing Environment

- Target: FalconStrix on Windows 11, Flask bound to 0.0.0.0:5001, isolated VMware network, MySQL 9.7 backing database.
- Attacker: Kali Linux (VMware), Burp Suite Community Edition (Repeater used to replay captured cookies).
- All testing performed exclusively against the researcher's own locally-hosted project.

This report documents a self-identified finding in a personal project, produced for educational / responsible-disclosure practice purposes.